National Railway Reservation and Management System — Implementation Plan

Oct 16, 2026 · @Sanskar Tiwari

Overview

We build the National Railway Reservation and Management System (NRRMS) with 6 groups of exactly 3 people (18 total) in one GitHub monorepo: React + Vite frontend, FastAPI backend, PostgreSQL 16 holding the schema and the business rules. Each group owns one module end to end, and each member owns one PostgreSQL object they explain in the viva.

How we differ from the reference project. The topic is the same, so we compete on depth in the database and on the "management" half of the title.

Area	Reference project	Our project
Module split	Auth, Search, Train details, Booking, Payment, PNR/Live, Admin	Accounts, Rail Network, Inventory & Fares, Reservation Engine, Wallet & Ticketing, Operations
Search	Direct trains only	Direct trains plus connecting journeys with one change of train
Seats	One berth sold once per train per date	Segment-wise: a berth freed at an intermediate station is resold for the rest of the route, enforced by a PostgreSQL exclusion constraint
Quotas	Quota code shown	Tatkal window (opens the day before travel) and concession rules enforced in SQL
Payment	Razorpay test mode	In-app wallet on a double-entry ledger; refunds credited automatically
After booking	PNR and simulated live map	Chart preparation before departure, staff console for delays and train cancellations, e-ticket PDF with QR

Area	Reference project	Our project
Data access	App-level checks	Row-level security: a passenger's queries can only see their own rows

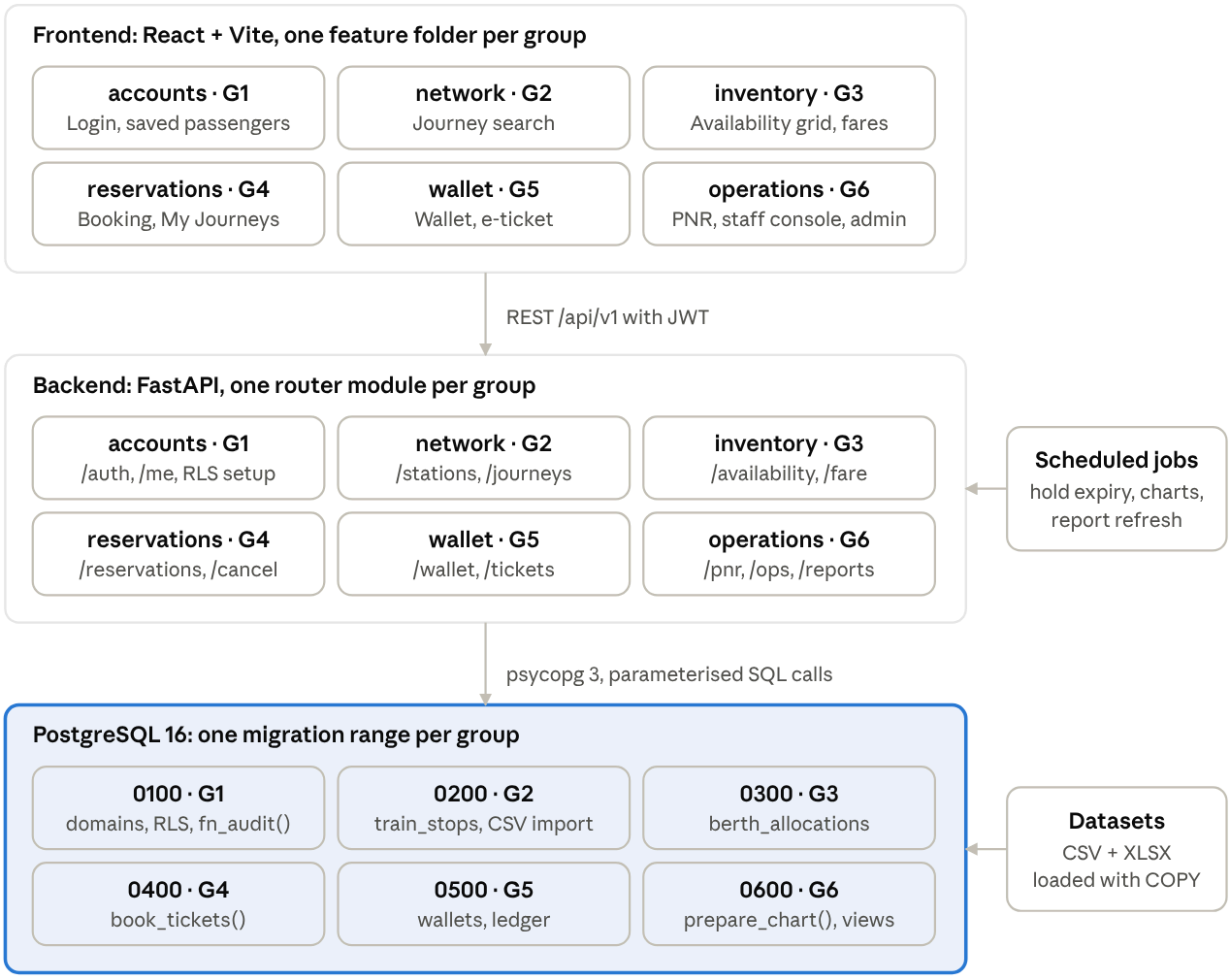
Data we load. `Train_details_22122017.csv` is the master timetable: 186,124 stop rows covering 11,113 trains and 8,151 stations. The workbook `Indian_Railway_Reservation_System.xlsx` supplies class, quota and status codes, concession rules, per-km fare slabs, zones, and about 20 sample PNRs for demo data. The 2015 CSV is older and smaller (69,006 rows, 2,810 trains), so it stays a fallback.

Out of scope: real payment gateways, live GPS tracking, and per-segment dynamic pricing.

Architecture and repository layout

One repo, three top-level folders, and one sub-folder per group in each layer, so two groups almost never edit the same file. The backend is a thin layer: routers validate input with Pydantic and call SQL functions; the database enforces the rules.

Pages call routers, routers call SQL; the rules live in PostgreSQL



NRRMS architecture · 3 layers, 6 modules each

Each group owns one column of chips through all three layers, so a feature is built top to bottom by the same three people.

```
nrrms/
├── .github/
|   ├── workflows/ci.yml          # lint + tests on every PR
|   ├── workflows/cd.yml          # deploy on push to develop / main
|   ├── CODEOWNERS                # each group reviews its own folders
|   └── pull_request_template.md
├── db/
|   ├── migrations/               # plain SQL, applied in filename order
|   |   ├── 0000_platform/        # extensions: pg_trgm, btree_gist, citext
(G1)
|   |   ├── 0100_g1_accounts/
```

```
|   |   ├── 0200_g2_network/
|   |   ├── 0300_g3_inventory/
|   |   ├── 0400_g4_reservations/
|   |   ├── 0500_g5_wallet/
|   |   └── 0600_g6_operations/
|   ├── seeds/                    # small fixtures used by CI
|   ├── tests/                    # SQL assertion scripts per group
|   ├── import/                   # CSV + XLSX loaders (G2, G3)
|   └── migrate.py                # ~40-line runner, records applied files
├── backend/
|   ├── app/main.py               # mounts every group's router
|   ├── app/core/                 # config, db pool, auth deps, RLS session
setup (G1)
|   ├── app/jobs/                 # scheduled jobs: hold expiry, chart prep,
report refresh
|   ├── app/modules/
|   |   ├── accounts/  network/  inventory/  reservations/  wallet/
operations/
|   |   |   └── router.py  schemas.py  service.py  sql.py
|   └── tests/<module>/           # pytest per group
├── frontend/
|   ├── src/api/client.ts         # axios instance + JWT interceptor (G1)
|   ├── src/components/           # shared UI kit
|   └── src/features/<module>/    # pages + hooks per group
├── docker-compose.yml            # postgres:16 + backend + frontend
└── .env.example
```

Rules that keep merges clean

- A group writes only inside its own folders. Shared folders (`app/core` , `app/jobs` , `src/components` , `0000_platform`) change by PR reviewed by G1.
- Migration numbers are reserved by range (G4 uses 0400–0499). A foreign key may point only to a lower range, so G4 can reference G3's berths but never the reverse.
- A merged migration is never edited. Fixes go in a new file.
- Each group exposes a router prefix (`/api/v1/reservations`) and publishes its request/response schemas on Day 2 with mock data, so frontends are never blocked.

Database design

32 tables and 2 materialized views in 3NF, each owned by one group; the owner writes the migration and answers questions on it. Foreign keys point only from a higher group number to a lower one.

Group	Tables it owns	Loaded from
G1 Accounts	`users`, `roles`, `user_roles`, `saved_passengers`, `audit_log`	Seed: 3 roles (passenger, staff, admin), demo users
G2 Rail Network	`zones`, `stations`, `trains`, `train_stops`, `train_running_days`, `staging_train_stops`	`Train_details_22122017.csv`; zones from the Stations sheet
G3 Inventory & Fares	`travel_classes`, `quotas`, `concession_rules`, `fare_slabs`, `train_consist`, `coaches`, `berths`, berth_allocations	Lookup_Tables and Fare_Matrix sheets; coaches and berths generated
G4 Reservation Engine	`reservations` (PNR), `reservation_passengers`, `cancellations`	PNR_Master sheet for demo data
G5 Wallet & Ticketing	`wallets`, `ledger_txns`, `ledger_entries`, `topups`, `refunds`, `e_tickets`	Created at runtime
G6 Operations	`train_runs`, `run_events`, `charts`, `chart_entries`, `mv_occupancy`, `mv_revenue`	Staff actions and scheduled jobs

Segment-wise berths (the core idea). A berth is sold for a range of stops, not for the whole train. PostgreSQL itself refuses two overlapping sales of the same berth on the same run, so no Python check can get it wrong:

```
CREATE EXTENSION IF NOT EXISTS btree_gist;

CREATE TABLE berth_allocations (
  allocation_id  BIGINT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
  -- passenger_id (FK) is added later by G4's migration 0401
  berth_id       BIGINT    NOT NULL REFERENCES berths,
  run_date       DATE      NOT NULL,          -- date the train leaves its
origin
  stops          INT4RANGE NOT NULL,          -- [boarding_seq,
alighting_seq)
  EXCLUDE USING gist (berth_id WITH =, run_date WITH =, stops WITH &&)
);
```

On a train with 10 stops, passenger A on stops [1,5) and passenger B on [5,10) can share berth S4-23; passenger C on [3,7) is rejected with an exclusion violation, and the reservation engine moves C to another berth or to RAC/WL.

Other design choices

- `train_stops(train_id, seq, station_id, arrival, departure, day_offset, distance_km)`, key `(train_id, seq)`. `day_offset` is not in the CSV; G2 derives it on import.
- Coaches and berths are generated per train from `train_consist` (for example 12 × SL, 5 × 3A), not imported.
- The CSV has no running days, so every train is assumed daily until staff edit `train_running_days`.
- Money is stored as `NUMERIC(10,2)`, never float.

Import pipeline (G2 owns it; CI runs it on a 500-row sample)

1. `COPY` the CSV into `staging_train_stops` as text (the file is Latin-1 and station names carry trailing spaces).
2. `INSERT … SELECT DISTINCT TRIM(...)` into `stations`, `trains`, then `train_stops`, with `ON CONFLICT DO NOTHING` so reruns are safe.
3. Compute `day_offset` with a window function, then truncate staging.
4. G3's Python loader (`openpyxl`) fills classes, quotas, concessions and fare slabs from the workbook; G4's loader turns PNR_Master into demo reservations.

Group-wise assignment

Each group has exactly three roles: Member 1 Frontend, Member 2 Backend, Member 3 Database. Each member also owns one PostgreSQL object they must explain, so SQL is spread across all 18 people. Each member writes the tests for their own part.

Group	Module	Group branch	Syllabus concepts its SQL covers
G1	Accounts and access	`group/g1-accounts`	DDL, domains, DCL roles, row-level security, triggers
G2	Rail network and journey planner	`group/g2-network`	Normalisation, self-joins, generated columns, trigram index, EXPLAIN, window functions
G3	Inventory and fares	`group/g3-inventory`	Range types, exclusion constraints, GiST, set-returning functions, `NOT EXISTS`
G4	Reservation engine	`group/g4-reservations`	Transactions, isolation levels, advisory locks, procedures, exception handling
G5	Wallet and ticketing	`group/g5-wallet`	Double-entry ledger, deferred constraint triggers, row locks, running totals, idempotent UPSERT
G6	Operations and analytics	`group/g6-operations`	PL/pgSQL cursors, views, materialized views, GROUPING SETS

G1 — Accounts and access

Goal: passengers, station staff and admins sign in with the right role, and the database itself stops a passenger reading anyone else's data. G1 also sets up the repo and CI on Day 1.

Member	Builds	SQL they explain
M1 Frontend	Sign-up, Login and Profile pages; Saved Passengers manager (up to 10 co-travellers); role-aware layout with passenger, staff and admin menus	`CREATE DOMAIN` types `mobile_no`, `pincode`, `email_addr` with CHECK rules, reused by every group's tables
M2 Backend	`/auth/signup`, `/auth/login`, `/auth/refresh`, `/me`, `/me/passengers`; bcrypt + JWT; `current_user` and `require_role()` dependencies; per-request `SET LOCAL app.user_id`; repo skeleton, `docker-compose.yml`, `migrate.py`, CI workflow	Row-level security: `ENABLE ROW LEVEL SECURITY` and a policy `USING (user_id = current_setting('app.user_id')::bigint)`; app roles `nrrms_app` and `nrrms_report` with GRANT/REVOKE
M3 Database	`users` (CITEXT email UNIQUE), `roles`, `user_roles`, `saved_passengers`, `audit_log`	`fn_audit()` trigger that stores only the changed columns as JSONB (diff of `to_jsonb(OLD)` and `to_jsonb(NEW)`); G4, G5 and G6 attach it

G2 — Rail network and journey planner

Goal: find direct trains and one-change connections between any two of 8,151 stations on a date.

Member	Builds	SQL they explain
M1 Frontend	Journey search with station autocomplete and date; results in two tabs, Direct and One change, showing total time and wait at the interchange; train timetable page	Trigram GIN index on `stations.name`; `EXPLAIN ANALYZE` before and after

Member	Builds	SQL they explain
M2 Backend	`GET /stations?q=`, `GET /journeys?from=&to=&date=` (direct + connections), `GET /trains/{no}/timetable`; input validation	Connecting-journey query: leg 1 and leg 2 meet at a common station, leg 2 leaves 30 minutes to 6 hours after leg 1 arrives, ranked by total time with `ROW_NUMBER()`
M3 Database	`zones`, `stations`, `trains`, `train_stops`, `train_running_days`; the CSV import	Import normalisation (186,124 rows to 8,151 stations), `day_offset` from a window over `LAG(departure)`, and a `GENERATED ALWAYS AS (...) STORED` minutes column used for time arithmetic

G3 — Inventory and fares

Goal: know exactly which berth is free between which two stops, and price any segment in any class and quota.

Member	Builds	SQL they explain
M1 Frontend	Availability grid (next 6 dates × classes, showing AVL / RAC / WL for the chosen segment), fare breakup modal, coach layout view of free berths for the segment	`availability_grid(train, from, to, start_date)`: a set-returning function using `generate_series` of dates `CROSS JOIN` classes
M2 Backend	`GET /trains/{no}/availability?from=&to=&date=&quota=`, `/fare`, `/coaches/{id}/layout`	`calculate_fare(...) RETURNS TABLE (base, reservation_fee, superfast, tatkal, concession, gst, total)`: slab by segment distance, concession % from `concession_rules`
M3 Database	`travel_classes`, `quotas`, `concession_rules`, `fare_slabs`, `train_consist`, `coaches`, `berths`, `berth_allocations`; workbook loader; berth generator	The exclusion constraint on `berth_allocations` and `free_berths(...)`: berths with `NOT EXISTS` an allocation whose `stops && int4range(from_seq, to_seq)`

G4 — Reservation engine

Goal: a reservation gives every passenger a berth, an RAC place or a WL number, or changes nothing, even when 20 users book the last berth at once.

Member	Builds	SQL they explain
M1 Frontend	Booking flow: pick saved passengers, berth preference, quota (General, Tatkal, Ladies, Senior), review, 10-minute hold timer, Tatkal countdown; My Journeys; cancel some or all passengers	`is_booking_open()` BEFORE INSERT trigger: 60-day advance window; Tatkal opens the day before travel at 10:00 for AC and 11:00 for non-AC (both configurable)
M2 Backend	`POST /reservations` (hold as PENDING_PAYMENT), `GET /reservations`, `GET /reservations/{pnr}`, `POST /reservations/{pnr}/cancel`; job releasing expired holds every minute; a test firing 20 parallel requests at one berth	`pg_advisory_xact_lock()` keyed on train, run date and class; two `psql` sessions showing READ COMMITTED vs REPEATABLE READ
M3 Database	`reservations` (10-digit PNR from a sequence), `reservation_passengers`, `cancellations`; adds `passenger_id` to `berth_allocations`; RLS policy on reservations	`book_tickets()` procedure: per passenger, try the preferred free berth, catch `exclusion_violation` and try the next, else RAC, else WL; `cancel_passenger()` frees the range and promotes RAC to CNF and WL to RAC

G5 — Wallet and ticketing

Goal: every rupee moves through a balanced ledger, a retried request never charges twice, and a paid PNR gets an e-ticket.

Member	Builds	SQL they explain
M1 Frontend	Wallet page (balance, mock UPI top-up screen, statement), pay-from-wallet step, e-ticket page with QR and PDF download, refund history	Statement with running balance: `SUM(amount) OVER (PARTITION BY wallet_id ORDER BY posted_at)`
M2 Backend	`GET /wallet`, `POST /wallet/topups` (Idempotency-Key header), `POST /reservations/{pnr}/pay`, `GET /tickets/{pnr}.pdf` (reportlab + qrcode), `GET /refunds`	Idempotent top-up: `UNIQUE (idempotency_key)` with `INSERT … ON CONFLICT DO NOTHING RETURNING`
M3 Database	`wallets` (`CHECK (balance >= 0)`), `ledger_txns`, `ledger_entries`, `topups`, `refunds`, `e_tickets`; trigger on G4's `cancellations` that creates the refund	Double-entry ledger: a `DEFERRABLE INITIALLY DEFERRED` constraint trigger checks each transaction sums to zero at COMMIT; `pay_reservation()` locks the wallet row `FOR UPDATE`; `refund_amount()` by hours before departure

G6 — Operations and analytics

Goal: staff run the railway (delays, cancellations, final charts) and admins see how full and how profitable each train is.

Member	Builds	SQL they explain
M1 Frontend	Public PNR status page; staff console (run board, delay entry, cancel a run, coach-wise chart); admin dashboard with Recharts: occupancy by class, revenue by day, WL-to-confirmed rate, busiest segments	`mv_revenue` with `GROUP BY GROUPING SETS ((day), (class), (day, class), ())` and `REFRESH MATERIALIZED VIEW CONCURRENTLY`

Member	Builds	SQL they explain
M2 Backend	`GET /pnr/{pnr}`; staff `/ops/runs`, `POST /ops/runs/{id}/delay`, `/cancel`, `/chart`; admin `/reports/*`; jobs for chart preparation at the cut-off and view refresh	`prepare_chart(run_id)`: PL/pgSQL with an explicit cursor over RAC/WL passengers in booking order, filling freed berths, then writing `chart_entries` and locking the run
M3 Database	`train_runs`, `run_events`, `charts`, `chart_entries`, `v_pnr_status`, `mv_occupancy`	`v_pnr_status` (six-table join) and the trigger that, when a run is set CANCELLED, cancels its reservations through G4's function so G5 refunds in full

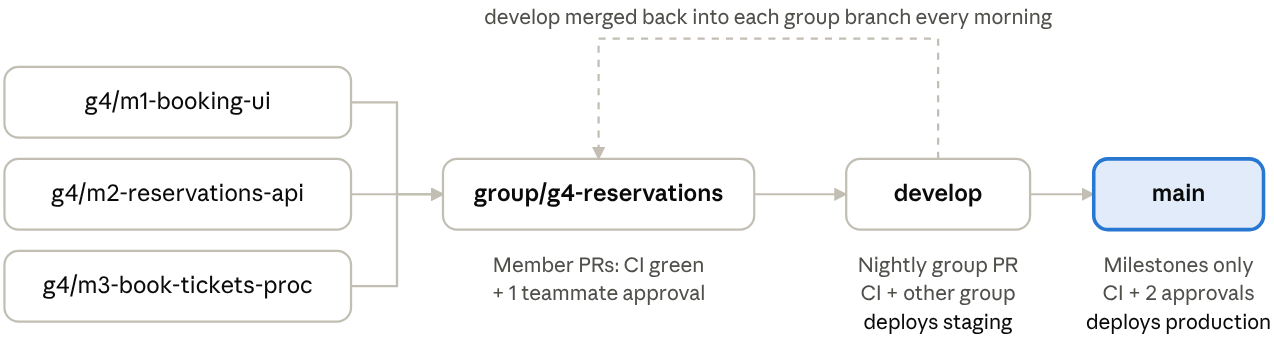
Contracts between groups. These are the only cross-group calls; anything else goes through an API.

From	To	Contract
G1	All	`current_user`, `require_role()`, `app.user_id` session setting, `fn_audit()`, domains
G2	G3, G4, G6	`train_id`, `station_id`, `train_stops.seq` with `day_offset`
G3	G4	`free_berths()`, `calculate_fare()`, `berth_allocations`; G4 writes allocations only inside `book_tickets()`
G4	G5	`reservations.total_fare`, `confirm_reservation(pnr)`, `cancellations` rows
G4, G5	G6	Read-only views; run cancellation goes through `cancel_passenger()`

Git branching strategy

Three levels of branch: each member works on a personal feature branch, merges into the group branch, and the group branch merges into `develop`; only `develop` merges into `main`. Nobody pushes directly to `develop` or `main`.

Code reaches main only through a group branch and develop



branch flow · G4 shown; every group works the same way

Branch	Created by	Example	Merges into	Needs to merge
`main`	G1, Day 1	—	— (deploys to production)	CI green + 2 approvals, only at milestones
`develop`	G1, Day 1	—	`main`	CI green + 1 approval from another group
Group branch	Each group	`group/g4-reservations`	`develop`	CI green + review by one member of a group that depends on it
Member branch	Each member	`g4/m2-reservations-api`, `g4/m3-book-tickets-proc`	Its group branch	CI green + 1 teammate approval

Daily routine for every member

1. Start of day: `git checkout group/g4-reservations && git pull`, then `git merge origin/develop` so the group sees other groups' merged work early.

2. Branch off: `git checkout -b g4/m3-book-tickets-proc`.
3. Small commits in Conventional Commit style: `feat(reservations): add book_tickets() with advisory lock`.
4. Push and open a PR into the group branch the same day; keep PRs under about 400 changed lines.
5. The group lead (rotate daily) opens one PR from the group branch to `develop` each evening once CI is green.

Branch protection (set by G1 on Day 1)

- `main`, `develop` and all `group/*` branches: require a pull request, require the CI checks to pass, block force-push and deletion.
- `CODEOWNERS` routes reviews automatically: `db/migrations/0400_g4_reservations/` and `backend/app/modules/reservations/` belong to G4, and so on.
- Squash-merge member PRs; use a merge commit for group → `develop` so each group's day of work stays visible in history.

PR checklist (in `pull_request_template.md`): screenshots for UI changes, example request and response for API changes, the migration file and a test for SQL changes, and a note on any other group's code this depends on.

CI/CD pipeline

GitHub Actions runs three checks on every pull request to a `group/*` branch, `develop` or `main`; a red check blocks the merge. Pushes to `develop` deploy to staging and pushes to `main` deploy to production. G1-M2 writes both workflows on Day 1; every group adds its own tests to them.

Check	What it runs	Fails when
`database`	Fresh `postgres:16` container → all migrations → sample import (500 CSV rows + workbook) → every `db/tests/*.sql` → migrations again	A migration errors, a number is reused or outside its group's range, a SQL assertion fails, or a rerun is not a no-op
`backend`	`ruff check`, then `pytest` against the migrated database	Lint error or failing test
`frontend`	`npm ci`, `npm run lint`, `vitest --run`, `vite build`	Lint, test or build error

```
# .github/workflows/ci.yml
name: CI
on:
  pull_request:
    branches: [main, develop, 'group/**']
  push:
    branches: [main, develop]

jobs:
  database:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env: { POSTGRES_PASSWORD: postgres, POSTGRES_DB: nrrms_test }
        ports: ['5432:5432']
        options: --health-cmd pg_isready --health-interval 5s --health-
retries 10
    env:
      DATABASE_URL: postgresql://postgres:postgres@localhost:5432/nrrms_test
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -r backend/requirements.txt
      - run: python db/migrate.py check      # numbering + ranges
      - run: python db/migrate.py up
      - run: python db/import/load_sample.py
      - run: for f in db/tests/*.sql; do psql "$DATABASE_URL" -v
```

```
ON_ERROR_STOP=1 -f "$f"; done
      - run: python db/migrate.py up         # must be a no-op

  backend:
    runs-on: ubuntu-latest
    services: { postgres: { image: 'postgres:16', env: { POSTGRES_PASSWORD:
postgres, POSTGRES_DB: nrrms_test }, ports: ['5432:5432'], options: '--
health-cmd pg_isready --health-interval 5s --health-retries 10' } }
    env:
      DATABASE_URL: postgresql://postgres:postgres@localhost:5432/nrrms_test
      JWT_SECRET: ci-only-secret
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with: { python-version: '3.12' }
      - run: pip install -r backend/requirements.txt
      - run: ruff check backend
      - run: python db/migrate.py up && pytest backend/tests -q

  frontend:
    runs-on: ubuntu-latest
    defaults: { run: { working-directory: frontend } }
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-node@v4
        with: { node-version: 20, cache: npm, cache-dependency-path:
frontend/package-lock.json }
      - run: npm ci
      - run: npm run lint
      - run: npx vitest --run
      - run: npm run build
```

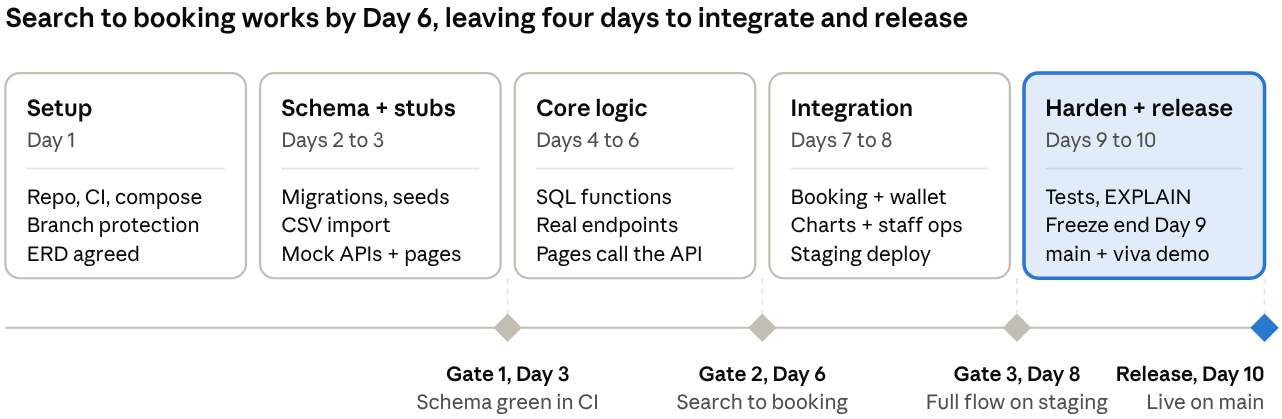
Deployment (`cd.yml`) — suggested free-tier hosts; any equivalent works.

Trigger	Database	Backend	Frontend
Push to `develop`	Neon branch `staging`: run `migrate.py up`	Render staging service via deploy hook	Vercel preview
Push to `main`	Neon `main`: run `migrate.py up` first	Render production service via deploy hook	Vercel production

Secrets (`DATABASE_URL` , `JWT_SECRET` , `RENDER_DEPLOY_HOOK`) live in GitHub Actions secrets and the hosts' environment settings; `.env.example` lists the names only. Migrations always run before the new backend starts, so the code never meets a schema older than itself.

Timeline and milestones

The plan runs 10 working days in five phases; a group's branch merges into `develop` for a phase only after it passes that phase's gate.



10-day plan · 5 phases, 4 gates

1. **Gate 1 (end of Day 3):** every group's migrations and seeds apply on a fresh database in CI, the CSV sample imports, and every router returns mock data matching its published schema.
2. **Gate 2 (end of Day 6):** on `develop` , a logged-in passenger can search direct and one-change journeys, see the availability grid and fare, and hold berths for a segment as PENDING_PAYMENT.
3. **Gate 3 (end of Day 8):** on staging, the full flow works: top up the wallet, pay, download the e-ticket, check the PNR; a berth freed at an intermediate stop is resold; cancelling refunds to the wallet and promotes RAC/WL; staff prepare a chart; the admin dashboard shows the sale.
4. **Release (Day 10):** code freeze at the end of Day 9 (bug fixes only), `develop` merged to `main` , production deployed, each member rehearses their viva checklist.

A 30-minute integration call at the end of each day: one rotating member per group demos what merged, and blockers between groups are settled there.

Conventions, definition of done and viva checklist

Conventions

- SQL: `snake_case`, plural table names, `<table>_id BIGINT GENERATED ALWAYS AS IDENTITY` keys, `created_at TIMESTAMPTZ DEFAULT now()` on every table, functions prefixed by verb (`get_`, `create_`, `calculate_`).
- One object per migration file, named `0412_g4_book_tickets_proc.sql`, with a header comment naming the author.
- API: everything under `/api/v1`, JSON errors as `{"detail": "...", "code": "SEAT_UNAVAILABLE"}`, list endpoints take `page` and `page_size`. FastAPI's `/docs` page is the API contract.
- Backend calls SQL through parameterised queries only (`%s` placeholders in psycopg); string-built SQL fails review.
- Frontend: one folder per feature, TanStack Query for server data, no API keys in React code.

Definition of done (per member PR)

- [ ] Migration applies on a fresh database and the rerun is a no-op
- [ ] At least one SQL test in `db/tests/` or pytest case covering the happy path and one failure
- [ ] Endpoint visible and tried in `/docs`, or screen screenshot in the PR
- [ ] CI green and one approval
- [ ] Module README updated: endpoints, tables, and the SQL objects this member owns

Viva checklist (every member)

- [ ] Show your SQL object in the migration file and run it live in `psql` or pgAdmin
- [ ] Explain which syllabus concept it demonstrates and why it lives in the database, not in Python
- [ ] Show the test that proves it works, including one failure case
- [ ] Show where the API or UI calls it
- [ ] Point to your commits on your member branch and your merged PR